ACCRA TECHNICAL UNIVERSITY

Department of Computer Science

BCS 404: Introduction to Data Science with Python

PROJECT REPORT

Exploratory Data Analysis, Statistical Analysis and Machine Learning using a Real-World Dataset (Titanic Dataset)

Student Name: Koomson Baafi Alexander

Index Number: 01258375B

Course Title: Data Science with Python

Lecturer: Dr. Joseph Dadzie

Academic Year: 2025/2026, Second Semester

(Deadline: Sunday, 19 July 2026)

Table of Contents

1. Introduction

This report presents a complete data science workflow applied to the Titanic dataset, undertaken as the project requirement for BCS 404: Introduction to Data Science with Python. The objective of the project is to demonstrate practical competence in data acquisition, data cleaning, exploratory data analysis, statistical analysis, and introductory machine learning using the Python data science stack (Pandas, NumPy, Matplotlib, Seaborn, and Scikit-Learn).

The sinking of the RMS Titanic on 15 April 1912 resulted in the loss of over 1,500 lives out of an estimated 2,224 passengers and crew. The Titanic dataset, made available through the Kaggle 'Titanic: Machine Learning from Disaster' competition, records demographic and travel information for 891 passengers together with a binary indicator of whether each passenger survived. This project uses that dataset to explore the factors associated with survival and to build a predictive classification model.

The remainder of this report is organized as follows: Section 2 describes the dataset; Section 3 outlines the methodology; Section 4 presents the results of data cleaning, visualization, statistical analysis, and the machine learning model; Section 5 discusses the findings, insights, limitations, and recommendations; and Section 6 concludes the report.

2. Dataset Description

The dataset was obtained from the Kaggle Titanic competition

(https://www.kaggle.com/competitions/titanic/data), specifically the train.csv file, which contains 891 passenger records across 12 columns.

Column	Description
PassengerId	Unique identifier for each passenger
Survived	Survival indicator (0 = Died, 1 = Survived) — target variable
Pclass	Ticket / passenger class (1 = 1st, 2 = 2nd, 3 = 3rd)
Name	Passenger's full name
Sex	Passenger's sex (male / female)
Age	Passenger's age in years
SibSp	Number of siblings/spouses aboard
Parch	Number of parents/children aboard
Ticket	Ticket number
Fare	Passenger fare paid
Cabin	Cabin number
Embarked	Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

Full details of dataset acquisition, including dimensions, column names, sample records, and data types, are presented in Section 4.1.

3. Methodology

3.1 Tools and Environment

The analysis was carried out in a Jupyter Notebook using Python 3, with Pandas and NumPy for data manipulation, Matplotlib and Seaborn for Visualization, and Scikit-Learn for machine learning.

3.2 Workflow

- Data Acquisition: Load the dataset and inspect its structure (dimensions, columns, data types, sample rows).
- Data Cleaning: Identify and treat missing values and duplicate records, with justification for each decision.
- Data Visualization: Produce six Visualizations (histogram, bar chart, boxplot, scatter plot, correlation heatmap, Pairplot) to explore relationships in the data.
- Statistical Analysis: Compute descriptive statistics, frequency distributions, and correlation coefficients; identify the strongest positive and negative correlations with survival.
- Machine Learning: Select predictor variables, split the data into training (80%) and testing (20%) sets, train a Logistic Regression classifier, and evaluate it using accuracy, a confusion matrix, and a classification report.

4. Results

4.1 Data Acquisition

The dataset was successfully loaded using pandas.read_csv(). It contains 891 rows and 12 columns, as summarised below.

Property	Value
Dataset dimensions	891 rows × 12 columns
Column names	PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
Data types	int64: PassengerId, Survived, Pclass, SibSp, Parch | float64: Age, Fare | object: Name, Sex, Ticket, Cabin, Embarked

4.2 Data Cleaning

Missing value detection revealed gaps in three columns:

Column	Missing Count	Missing %	Action Taken
Age	177	19.9%	Imputed with the median age
Cabin	687	77.1%	Column dropped (too sparse to impute reliably)
Embarked	2	0.2%	Imputed with the mode (most frequent port)

No duplicate rows were detected in the dataset (0 duplicates out of 891 records), so no rows were removed on that basis. After cleaning, the working dataset contains 891 rows and 11 columns (Cabin removed), with no remaining missing values in Age or Embarked.

4.3 Data Visualisation

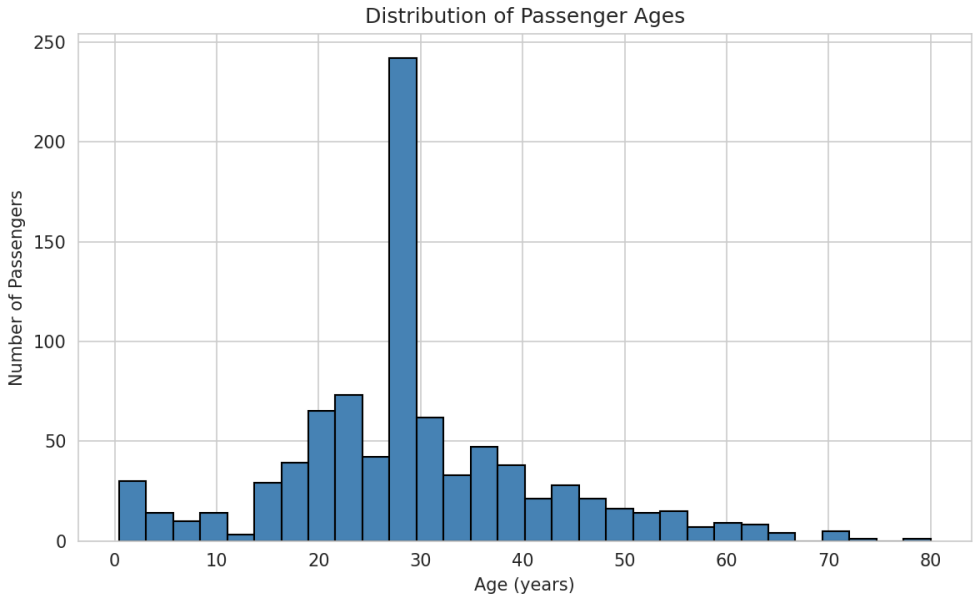

Figure 1: Histogram of Passenger Ages

The age distribution is right-skewed and concentrated between 20 and 40 years, with a visible spike near the median-imputed value (~28 years).

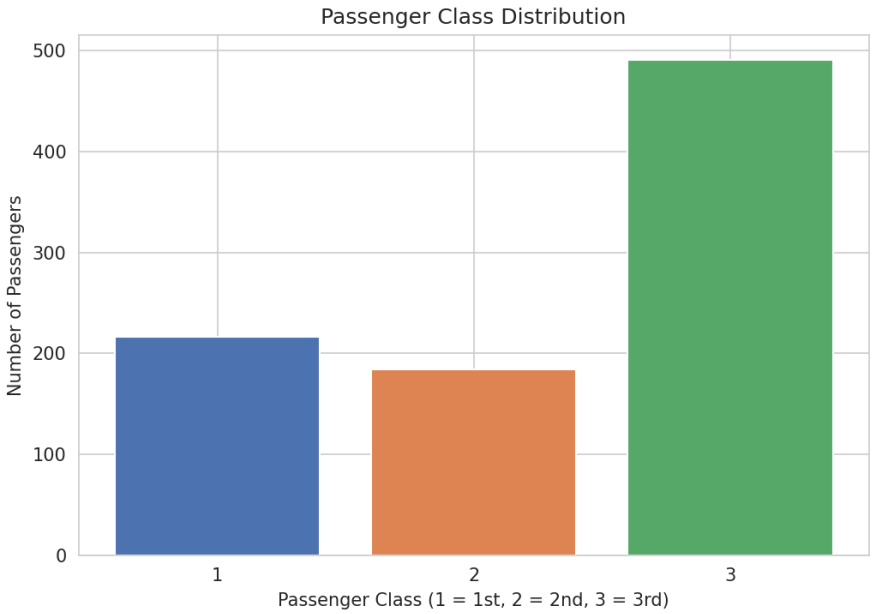

Figure 2: Passenger Class Distribution

Third class passengers form the largest group, followed by first class and then second class, reflecting the Titanic's actual passenger mix.

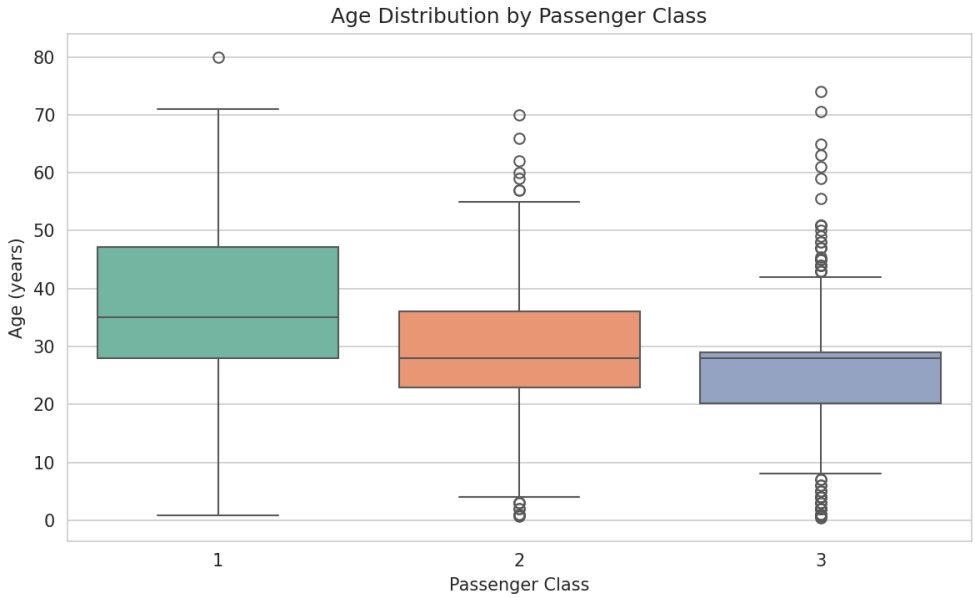

Figure 3: Boxplot of Age by Passenger Class

Median age decreases with passenger class: first class passengers are, on average, older than second- and third-class passengers.

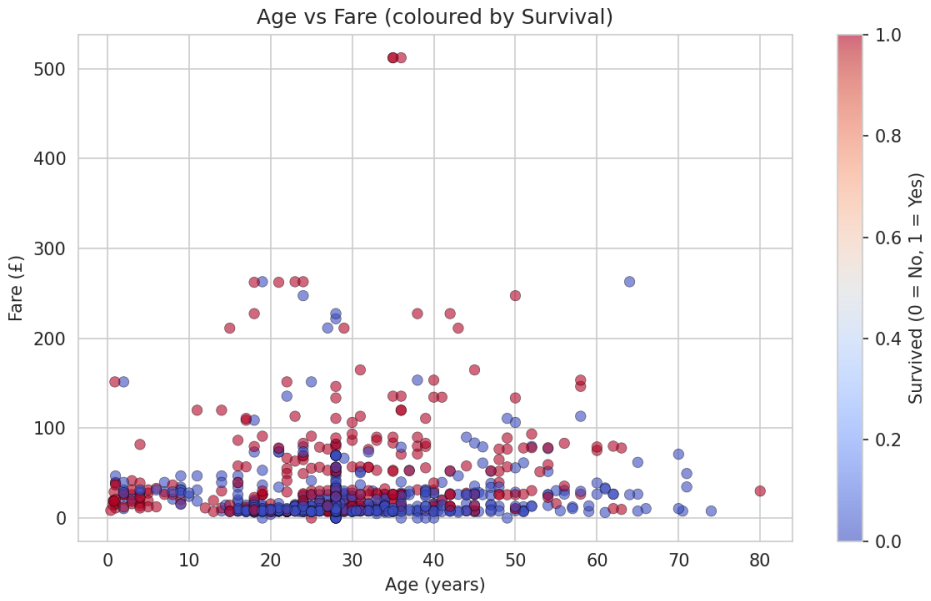

Figure 4: Scatter Plot of Age versus Fare (coloured by survival)

No strong linear relationship exists between age and fare, but passengers who paid higher fares appear more concentrated among survivors.

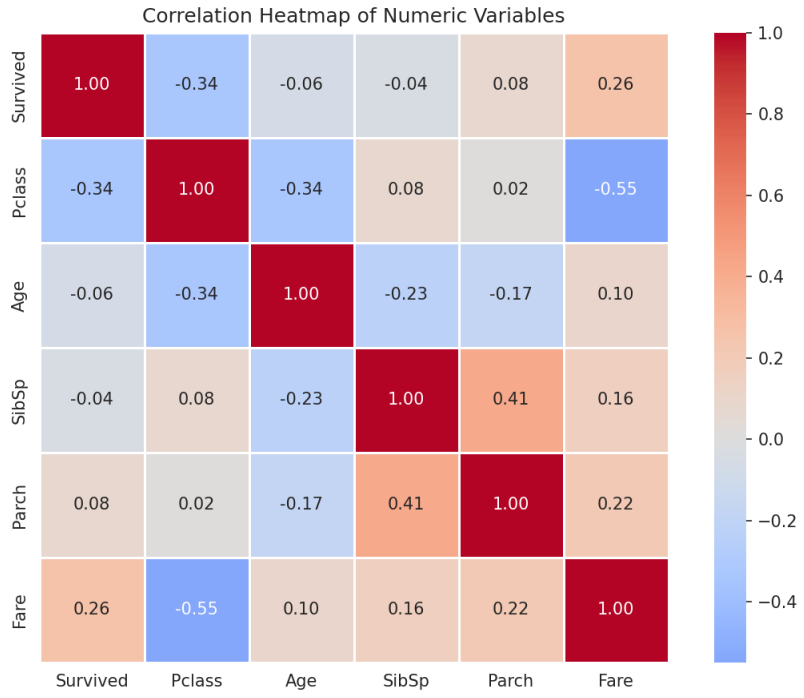

Figure 5: Correlation Heatmap of Numeric Variables

Pclass shows the strongest negative correlation with Survived (-0.34), and Fare shows the strongest positive correlation with Survived (0.26) among the numeric variables considered.

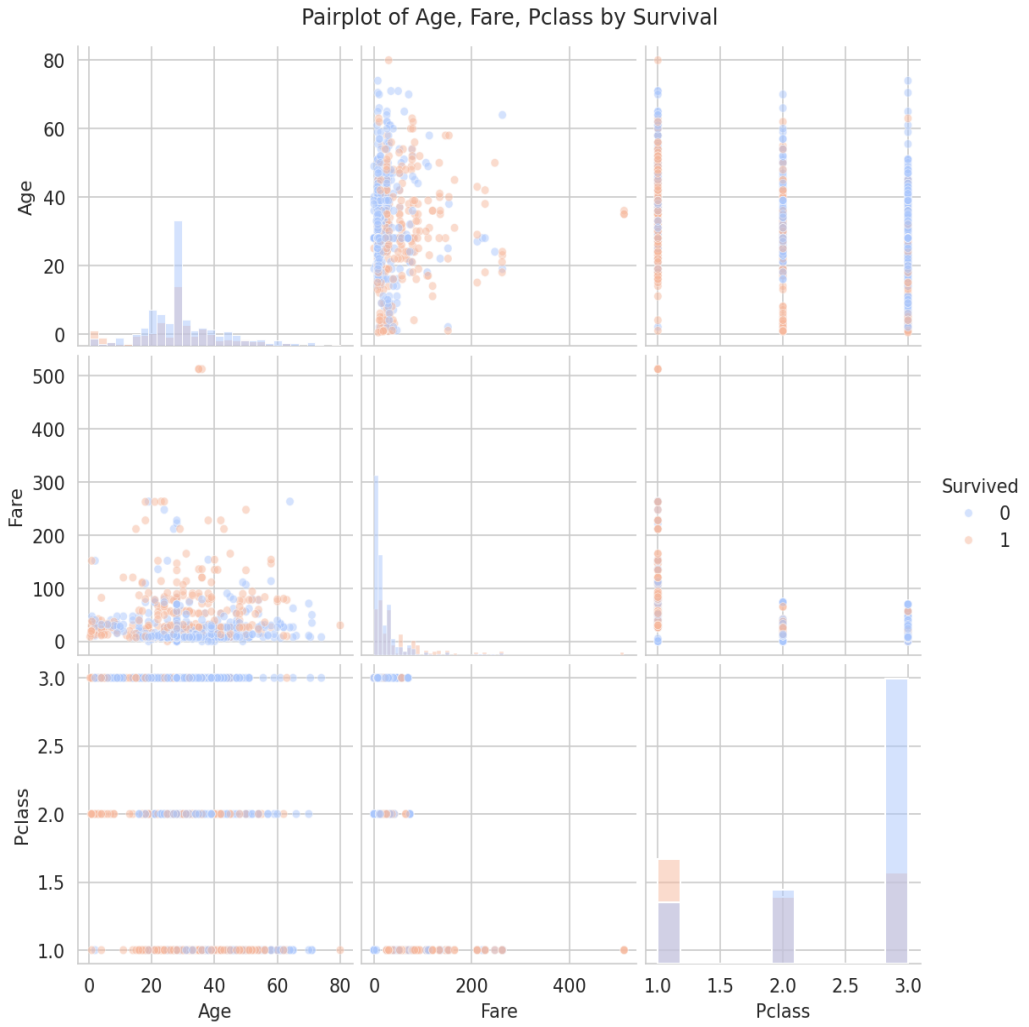

Figure 6: Pairplot of Survived, Age, Fare, and Pclass

Survivors are concentrated in lower Pclass values and, to a lesser extent, higher fares, while non-survivors are spread more broadly across all classes.

4.4 Statistical Analysis

Descriptive statistics for the key numeric variables are summarised below.

Statistic	Age	Fare	SibSp	Parch
Mean	29.4	32.2	0.52	0.38
Std. Dev.	13.0	49.7	1.10	0.81
Min	0.42	0.00	0	0
Max	80.0	512.3	8	6

Frequency distribution: 342 of 891 passengers survived (38.4%) while 549 died (61.6%). By class, 3rd class accounts for the largest share of passengers, followed by 1st and 2nd class. By sex, there were more male than female passengers overall, though female passengers survived at a substantially higher rate. The majority of passengers embarked from Southampton (S).

Correlation analysis (numeric variables vs. Survived):

Variable	Correlation with Survived
Fare	+0.257 (strongest positive)
Parch	+0.082
SibSp	-0.035
Age	-0.065
Pclass	-0.338 (strongest negative)

Three important statistical findings:

- Passenger class is the strongest numeric predictor of survival: Pclass correlates at -0.338 with Survived, confirming that passengers in lower classes (3rd class) had markedly lower survival rates.

- Fare, closely related to class and wealth, correlates positively with survival (+0.257), reinforcing that economic status influenced access to lifeboats.

- Family-size indicators (SibSp, Parch) show weak correlations individually, suggesting family size alone was not a decisive survival factor, though qualitative patterns (e.g., 'women and children first') are better captured by the Sex variable than by these numeric fields.

4.5 Machine Learning Model

Predictor variables: Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked. Sex and Embarked were label-encoded to numeric form. The data was split into an 80% training set (712 records) and a 20% testing set (179 records), using stratified sampling to preserve the survival class ratio.

A Logistic Regression classifier (scikit-learn, max_iter=1000) was trained on the training set and evaluated on the held-out test set.

Metric	Value
Accuracy	80.45%
Baseline accuracy (predict 'died' for all)	61.62%

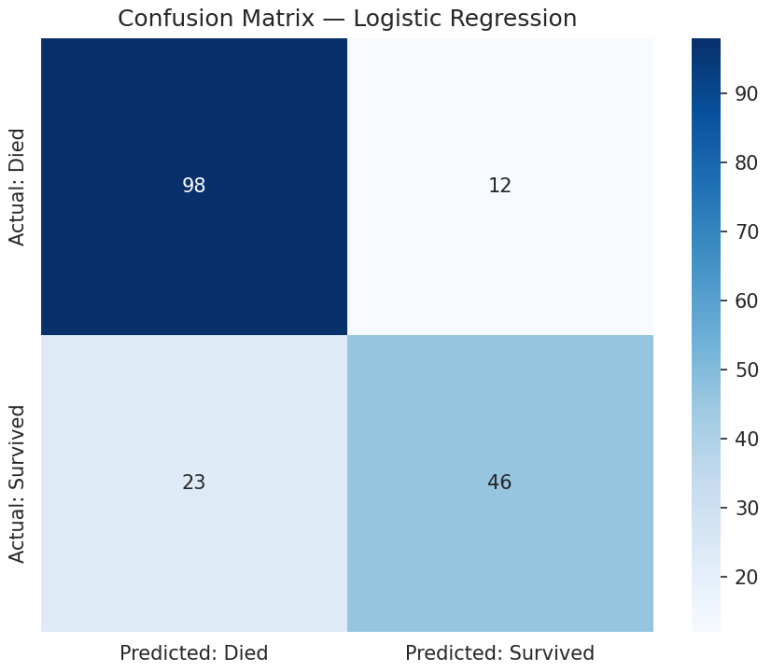

Figure 7: Confusion Matrix — Logistic Regression

Confusion matrix breakdown: of 110 passengers who actually died, 98 were correctly predicted as died and 12 were incorrectly predicted as survived. Of 69 passengers who actually survived, 46 were correctly predicted as survived and 23 were incorrectly predicted as died.

Classification report:

Class	Precision	Recall	F1-score	Support
Died	0.81	0.89	0.85	110
Survived	0.79	0.67	0.72	69
Accuracy			0.80	179

Model performance discussion: The model achieves 80.45% accuracy, roughly 19 percentage points above the 61.6% baseline achieved by always predicting 'died'. Precision and recall are higher for the 'Died' class (0.81 / 0.89) than the 'Survived' class (0.79 / 0.67), meaning the model is somewhat more conservative about predicting survival, missing about a third of actual survivors (23 of 69). This is consistent with the class imbalance in the training data (61.6% died vs. 38.4% survived). Overall, the model demonstrates that a small set of readily available features — particularly class, sex, and fare — can predict Titanic survival substantially better than chance.

5. Discussion

5.1 Major Findings

This project set out to explore the Titanic dataset and to build a model capable of predicting passenger survival. The exploratory and statistical analysis converged on a consistent narrative: survival on the Titanic was not random but was strongly shaped by socio-economic status and, implicitly, by sex and age. Passengers travelling in first class were considerably more likely to survive than those in third class, and this pattern was echoed in the strong negative correlation between Pclass and Survived (-0.338) and the positive correlation between Fare and Survived (+0.257). The visualisations reinforced this picture: the class distribution bar chart showed that third class passengers, who fared worst in survival terms, also made up the majority of the ship's population, while the boxplot of age by class showed that first class passengers tended to be older and, by extension, likely to be wealthier, established individuals more able to secure passage to lifeboats.

5.2 Statistical Insights

The frequency distribution confirmed the well-known overall survival rate of approximately 38%, with the remaining 62% of passengers perishing in the disaster. The correlation analysis added precision to the qualitative narrative often told about the Titanic: while Pclass and Fare emerged as the most informative numeric predictors, variables such as Age, SibSp, and Parch showed only weak linear relationships with survival, indicating that family size and age alone do not fully explain who survived and who did not. This suggests that categorical factors not captured in the correlation matrix — most notably Sex, which recorded markedly different survival rates between male and female passengers in the frequency tables — were at least as influential as the numeric variables, and arguably more so, echoing the historical 'women and children first' evacuation protocol.

5.3 Machine Learning Results

The Logistic Regression model, trained on seven features including the label-encoded Sex and Embarked variables, achieved 80.45% accuracy on unseen test data, a substantial improvement over the 61.6% baseline of always predicting non-survival. The precision and recall figures reveal an asymmetry: the model is more accurate at identifying passengers who died (89% recall) than those who survived (67% recall). In practical terms, the model is conservative — it tends to under-predict survival, which is a reasonable trade-off in many contexts but would need to be considered carefully if this model were ever used to inform decisions rather than purely for retrospective analysis. The F1-scores of 0.85 for the 'Died' class and 0.72 for the 'Survived' class summarise this imbalance in predictive quality between the two outcomes.

5.4 Limitations of the Study

- Missing data: Nearly 20% of Age values and over 77% of Cabin values were missing. Median imputation for Age, while a reasonable default, cannot fully recover the true distribution of ages, and dropping Cabin discards potentially useful information about a passenger's location on the ship, which may have influenced access to lifeboats.

- Model simplicity: Logistic Regression is a linear model and cannot capture complex non-linear interactions between features (for example, how class and sex may interact differently across age groups) as effectively as ensemble methods such as Random Forests or Gradient Boosting.

- Small dataset: With only 891 records split into training and testing sets, the model was trained on 712 rows and evaluated on just 179, which limits the statistical robustness of the reported accuracy and leaves it sensitive to the specific train/test split used.

- Feature engineering: No additional features were engineered from existing fields (e.g., extracting titles from Name, or combining SibSp and Parch into a family-size feature), which is a common step in Titanic modelling exercises known to improve predictive performance.

- Class imbalance: The dataset is moderately imbalanced (61.6% died vs. 38.4% survived), which likely contributes to the model's weaker recall on the minority 'Survived' class.

5.5 Recommendations

- Engineer additional features, such as extracting passenger titles (Mr., Mrs., Miss, Master) from the Name field and constructing a FamilySize variable from SibSp and Parch, to potentially improve model performance.

- Experiment with more flexible models, such as Random Forest or Gradient Boosting classifiers, and compare their accuracy, precision, and recall against the Logistic Regression baseline established here.

- Apply techniques for handling class imbalance, such as class weighting or resampling, to improve recall on the 'Survived' class.

- Use cross-validation rather than a single train/test split to obtain a more robust estimate of model performance and reduce sensitivity to how the data happens to be partitioned.

- Investigate the Cabin field more closely (e.g., extracting deck letters from the non-missing values) rather than dropping it outright, since cabin location may carry genuine survival-relevant information.

6. Conclusion

This project applied a complete data science workflow — data acquisition, cleaning, visualisation, statistical analysis, and machine learning — to the Titanic dataset using Python. The analysis confirmed that passenger class and fare were the strongest numeric predictors of survival, with third-class and lower-fare passengers facing substantially worse survival outcomes than their first-class counterparts. A Logistic Regression model built on seven features achieved 80.45% test accuracy, meaningfully outperforming a naive baseline and demonstrating that survival on the Titanic can be predicted with reasonable confidence from basic demographic and travel information. While the model has limitations — including its linear nature, the small dataset size, and simplifications made during data cleaning — it nonetheless offers a clear, data-driven confirmation of the historically documented inequalities in survival outcomes aboard the Titanic. Future work incorporating richer feature engineering and more flexible models could further improve predictive performance.

7. References

Kaggle. (n.d.). Titanic: Machine Learning from Disaster. Retrieved from https://www.kaggle.com/competitions/titanic/data

Pandas Development Team. (2024). pandas documentation. Retrieved from https://pandas.pydata.org/docs/

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95.

8. Appendix: Python Code

The full, executable code used for this project (as run in the accompanying Jupyter Notebook, Titanic_Data_Science_Project.ipynb) is provided below.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)
pd.set_option("display.max_columns", None)

print("Libraries imported successfully.")

df = pd.read_csv("titanic.csv")
print("Dataset loaded successfully.")

# Dataset dimensions
print(f"Dataset dimensions (rows, columns): {df.shape}")

# Column names
print("Column names:")
print(list(df.columns))

# First five observations
df.head()

# Data types
df.dtypes

missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df) * 100).round(2)
missing_summary = pd.DataFrame({"Missing Count": missing, "Missing %": missing_pct})
missing_summary = missing_summary[missing_summary["Missing Count"] > 0].sort_values("Missing
Count", ascending=False)
missing_summary

df_clean = df.copy()

# Age: impute with the median (robust to outliers, preserves distribution shape better than the mean)
df_clean["Age"] = df_clean["Age"].fillna(df_clean["Age"].median())
```

```
# Embarked: impute with the mode, since only 2 values are missing out of 891
df_clean["Embarked"] = df_clean["Embarked"].fillna(df_clean["Embarked"].mode()[0])

# Cabin: over 77% missing — the column itself is not reliably usable, so it is dropped
# rather than imputed, which would introduce substantial noise/bias
df_clean = df_clean.drop(columns=["Cabin"])

print("Remaining missing values per column:")
print(df_clean.isnull().sum())

duplicate_count = df_clean.duplicated().sum()
print(f"Number of duplicated rows: {duplicate_count}")

if duplicate_count > 0:
   df_clean = df_clean.drop_duplicates()
   print(f"Duplicates removed. New shape: {df_clean.shape}")
else:
   print("No duplicate rows were found; no removal necessary.")

plt.figure(figsize=(8, 5))
plt.hist(df_clean["Age"], bins=30, color="steelblue", edgecolor="black")
plt.title("Distribution of Passenger Ages")
plt.xlabel("Age (years)")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.savefig("figures/1_age_histogram.png", dpi=150)
plt.show()

plt.figure(figsize=(7, 5))
class_counts = df_clean["Pclass"].value_counts().sort_index()
plt.bar(class_counts.index.astype(str), class_counts.values, color=["#4C72B0", "#DD8452", "#55A868"])
plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class (1 = 1st, 2 = 2nd, 3 = 3rd)")
plt.ylabel("Number of Passengers")
plt.tight_layout()
plt.savefig("figures/2_class_bar.png", dpi=150)
plt.show()

plt.figure(figsize=(8, 5))
sns.boxplot(x="Pclass", y="Age", data=df_clean, hue="Pclass", palette="Set2", legend=False)
plt.title("Age Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Age (years)")
plt.tight_layout()
plt.savefig("figures/3_age_by_class_boxplot.png", dpi=150)
plt.show()

plt.figure(figsize=(8, 5))
```

```
scatter = plt.scatter(df_clean["Age"], df_clean["Fare"],
                      c=df_clean["Survived"], cmap="coolwarm", alpha=0.6, edgecolor="k", linewidth=0.3)
plt.title("Age vs Fare (coloured by Survival)")
plt.xlabel("Age (years)")
plt.ylabel("Fare (£)")
plt.colorbar(scatter, label="Survived (0 = No, 1 = Yes)")
plt.tight_layout()
plt.savefig("figures/4_age_vs_fare_scatter.png", dpi=150)
plt.show()

numeric_cols = ["Survived", "Pclass", "Age", "SibSp", "Parch", "Fare"]
corr_matrix = df_clean[numeric_cols].corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0, square=True,
linewidths=0.5)
plt.title("Correlation Heatmap of Numeric Variables")
plt.tight_layout()
plt.savefig("figures/5_correlation_heatmap.png", dpi=150)
plt.show()

pairplot_fig = sns.pairplot(df_clean[["Survived", "Age", "Fare", "Pclass"]], hue="Survived",
                            palette="coolwarm", diag_kind="hist", plot_kws={"alpha": 0.5, "s": 20})
pairplot_fig.fig.suptitle("Pairplot of Age, Fare, Pclass by Survival", y=1.02)
pairplot_fig.savefig("figures/6_pairplot.png", dpi=150)
plt.show()

df_clean.describe()

for col in ["Survived", "Pclass", "Sex", "Embarked"]:
    print(f"--- {col} ---")
    print(df_clean[col].value_counts())
    print(df_clean[col].value_counts(normalize=True).round(3) * 100, "\n")

corr_matrix

# Identify strongest positive and negative correlations with Survived (excluding self-correlation)
corr_with_target = corr_matrix["Survived"].drop("Survived").sort_values(ascending=False)
print("Correlation of each variable with Survived (sorted):")
print(corr_with_target)

strongest_positive = corr_with_target.idxmax()
strongest_negative = corr_with_target.idxmin()
print(f"\nStrongest positive correlation with Survived: {strongest_positive}
({corr_with_target.max():.3f})")
print(f"Strongest negative correlation with Survived: {strongest_negative}
({corr_with_target.min():.3f})")
```

```
ml_df = df_clean.copy()

le_sex = LabelEncoder()
ml_df["Sex"] = le_sex.fit_transform(ml_df["Sex"])        # female=0, male=1

le_embarked = LabelEncoder()
ml_df["Embarked"] = le_embarked.fit_transform(ml_df["Embarked"])

features = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare", "Embarked"]
X = ml_df[features]
y = ml_df["Survived"]

print("Feature matrix shape:", X.shape)
X.head()

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y
)
print(f"Training set size: {X_train.shape[0]} rows")
print(f"Testing set size: {X_test.shape[0]} rows")

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
print("Model trained successfully.")

y_pred = model.predict(X_test)
print("Predictions generated for the test set.")

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
        xticklabels=["Predicted: Died", "Predicted: Survived"],
        yticklabels=["Actual: Died", "Actual: Survived"])
plt.title("Confusion Matrix — Logistic Regression")
plt.tight_layout()
plt.savefig("figures/7_confusion_matrix.png", dpi=150)
plt.show()

report = classification_report(y_test, y_pred, target_names=["Died", "Survived"])
print("Classification Report:")
print(report)
```